SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science & Engineering

CO1 - ASSESSMENT TOOL - 1

Short Answer Test

Course Code	**CSA3717**	**Course Title**	**Software Testing**
CO Assessed	**CO1**	**Assessment**	**Assessment Tool 2: Short Answer Test**
Weightage	**50% of CIA**	**Total Marks**	**100**
CO1 Statement	*Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.* **(BL3)**		
Student Name	**M Sandhya**	**Reg. No.**	**192312421**
Section/Batch	**Slot A /2023**	**Date**	**23-10-2026**

Code of Conduct:

I, Anchal Kumari, 192312255, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

M Sandhya

SIMATS ENGINEERING

Short Answer Test

COURSE NAME: Software Testing **COURSE CODE: CSA3717**

COURSE OUTCOME COVERED

CO1: Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.

Assessment Tool Weightage: 30%

Question 1

A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.

Answer: Concept: Testing is an engineering discipline, not an ad hoc activity. Like any engineering task, it needs planning, design, execution, measurement and improvement. Random testing depends on the tester's mood and memory, so results cannot be repeated or measured, and defects get missed.

Redesigned approach:

1. **Test planning: define scope, objectives, risks, resources, schedule and entry/exit criteria.**
2. **Requirement analysis: identify testable conditions from each requirement.**
3. **Test design: apply techniques such as equivalence partitioning, boundary value analysis, decision tables and state-transition testing to create test cases with defined inputs, preconditions and expected results.**
4. **Test execution: run the cases, compare actual and expected results, and log defects in a standard format.**
5. **Measurement and closure: track coverage, defect density and pass/fail rates, then review lessons learned.**

How structured design improves quality:

- **It gives traceability, since every requirement maps to at least one test case.**
- **It is repeatable, so two testers produce the same results.**
- **It reduces gaps and duplication, because techniques systematically cover valid, invalid and boundary inputs.**
- **It makes coverage measurable, so the manager can see how much has been tested.**
- **It finds more defects with fewer tests.**

Example: For a signup form with an age field (18–60), random testing might try 25 and 30. Structured design adds 17, 18, 60, 61, a blank value and a letter, which find the defects that random values miss.

Question 2

A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.

Answer: Concept: A testing process is a defined set of activities, roles, inputs and outputs that the whole organization follows. Banking is a real-time, high-risk domain where failures mean financial loss, regulatory penalties and lost trust.

Phases of a defined process:

- **Test planning and control:** scope, strategy, risks and schedule.
- **Test analysis and design:** test conditions, cases and data.
- **Test environment setup:** a production-like environment with masked data.
- **Test execution:** run tests, log defects, retest, run regression.
- **Evaluating exit criteria:** for example, no open critical defects and 95% of cases passed.
- **Closure:** archive testware and record lessons learned.

Benefits with examples:

- **Fewer production failures:** a standard test for "transfer amount greater than balance" catches overdraft defects.
- **Consistency:** every team tests interest calculation, EMI and statements the same way.
- **Early defect detection:** reviews catch rule errors before coding.
- **Regression safety:** a defined regression cycle ensures a change to the ATM module does not break net banking.
- **Audit and compliance:** documented evidence supports regulators such as the RBI.
- **Lower cost and risk:** defects cost far less to fix before release than after.

Conclusion: A defined process turns testing from a person-dependent activity into a controlled, measurable one, which is essential for banking software.

Question 3

During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.

Answer: Concept: Integration and system-level defects appear when testing is done only at the end or inconsistently. Testing must be built into every SDLC phase. The **V-model** shows each development phase paired with a test level.

SDLC phase	Testing activity
Requirements	Requirement review, acceptance test planning
System design	System test planning
High-level design	Integration test planning

SDLC phase	Testing activity
Low-level design/coding	Unit test design, code review, unit testing
Integration	Integration testing (cart ↔ payment ↔ inventory ↔ order)
System build	System testing (functional, performance, security)
Delivery	User acceptance testing (UAT)

Applying to e-commerce:

- Unit-test the discount, tax and price modules.
- Integration-test the data flow from cart to payment gateway to order confirmation to inventory update.
- System-test the whole platform for load during sales, cross-browser behavior and security.
- Run regression tests whenever a module changes.

How process discipline ensures quality:

- Each phase has **entry and exit criteria**, so poor-quality work does not move forward.
- Defects are found **in the phase where they are introduced**, which keeps cost low.
- **Interfaces are tested deliberately**, which reduces integration defects.
- Documentation and metrics make quality **predictable and improvable**.

Question 4

A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.

Answer: Definitions:

- **Error (mistake):** a human action that produces an incorrect result, such as a developer misunderstanding the password rule.
- **Defect (bug/fault):** a flaw in the code, design or document caused by an error. It is the incorrect statement in the program.
- **Failure:** the deviation of the system from expected behavior, seen when a defect is executed.

Relationship: Error → Defect → Failure. An error introduces a defect. When the defective code is executed under certain conditions, a failure appears. Not every defect causes a failure, since the faulty code may never run, and some failures come from environmental conditions.

Classification of the login issue:
Expected output: “Login successful” for valid credentials. Actual output: “Invalid password”. This mismatch seen during validation is a **failure**.

Tracing it back:

- **Failure:** valid users are rejected.
- **Defect:** the code compares the password case-insensitively against a case-sensitive stored hash, or compares the wrong variable.
- **Error:** the developer misread the requirement or made a coding slip.

The team should therefore log it as a **defect that caused a failure**, root-cause it to the error, fix the defect, and retest.

Question 5

A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.

Answer: Principle: Early testing (shift-left). Testing activities should start as early as possible in the SDLC and be focused on defined objectives.

Why it addresses the problem: The cost of fixing a defect grows sharply the later it is found. A requirement defect fixed at the requirements stage costs a small amount. The same defect found in production may need redesign, recoding, retesting, redeployment and customer compensation.

How to apply it:

- Review requirements and designs (static testing) for ambiguity and inconsistency.
- Write test cases while requirements are still being finalized.
- Do unit testing and code reviews during development.
- Use continuous integration so defects surface within hours.

How it improves effectiveness:

- Defects are found when they are cheapest to fix.
- It prevents defects from propagating into design and code.
- It reduces rework and schedule risk.
- It improves requirement quality, since testers question ambiguity.

Example: A requirement says “Discount applies to large orders.” A tester asks, “How large?” and the answer is defined as ₹5,000. This takes minutes. If the vagueness had reached coding, different modules could have used different thresholds, causing defects found only in system testing.

Question 6

During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.

Answer: Problem: Unnoticed ambiguities become implementation defects. In healthcare these can affect patient safety.

How a tester contributes:

1. **Reviewing requirements** for ambiguity ("quickly", "appropriate dosage"), incompleteness, contradictions and untestable statements.
2. **Asking clarifying questions**, for example about valid ranges, units, error handling and user roles.
3. **Defining acceptance criteria** so every requirement has a clear pass/fail condition.
4. **Building a requirement traceability matrix (RTM)** linking requirements to test cases.
5. **Identifying risks** such as patient safety and data privacy, and prioritizing them.
6. **Starting test design early**, because writing test cases often reveals gaps.

Importance of early involvement:

- Testers bring an independent, "how could this fail?" viewpoint.
- Defects are prevented, not just detected.
- It saves cost and time, since requirement defects are the most expensive when found late.
- It improves communication and shared understanding between business, developers and testers.

Example: A requirement says "The system shall alert on high dosage." The tester asks what counts as high, the unit (mg or ml), and the patient category (adult or child). This prevents a serious dosage-calculation defect.

Question 7

A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.

Answer: Defects can originate when stakeholders provide incomplete rules, analysts interpret them incorrectly, designers omit cases, developers implement assumptions, or testers lack clear expected results. In payroll, ambiguity about overtime, tax, leave, or rounding may cause incorrect salaries. Use stakeholder reviews, examples, decision tables, a glossary, measurable acceptance criteria, and requirement traceability. Clear, agreed requirements give developers and testers the same understanding and prevent avoidable defects.

How defects originate across the lifecycle:

- **Requirements phase (the largest source):** ambiguous, missing, conflicting or incorrect requirements. For example, "Calculate overtime" without the rate, the threshold or whether holidays count.
- **Design phase:** the designer interprets the vague requirement wrongly, giving wrong data models and algorithms.
- **Coding phase:** the developer codes the wrong design, or makes logic, syntax, boundary and data-handling mistakes.
- **Testing phase:** testers base test cases on the same misunderstanding, so defects are not detected.
- **Maintenance:** changes cause side effects and regressions.

Defect propagation: A requirement defect is multiplied through design, code and tests. This is why unclear requirements dominate defect causes in payroll, where tax slabs, deductions, bonuses and leave rules are complex.

How requirement clarity prevents this:

- Write requirements that are **clear, complete, consistent, unambiguous and testable**.
- Use examples (for instance, "Salary ₹50,000 with 10 hours overtime → pay ₹X").
- Hold **formal reviews and inspections** with business, developers and testers.
- Maintain a **glossary** and use decision tables for rules.
- Keep **traceability** from requirement to design, code and test.

Clear requirements give all stakeholders the same interpretation, which removes the root cause.

Question 8

A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.

Answer: This is a boundary-related input-validation defect, potentially causing a robustness or reliability failure. Boundary value analysis tests values at and around limits, because errors often occur at the edges of allowed ranges. If an age field permits 1–120, test 0, 1, 2, 119, 120, and 121, along with empty or malformed values where relevant. These tests can reveal off-by-one errors, missing checks, and crashes before release.

Classification: This is a **functional defect** in **input validation/boundary handling**. It shows up as a **failure** (crash) when inputs sit at the limits of the valid range. The cause is usually an off-by-one error in a comparison (< instead of <=) or a missing check on minimum and maximum values.

Justification:

- The behavior deviates from the specified functionality (the system should accept or reject boundary values gracefully, not crash).
- It is triggered by a specific input class (boundaries).
- It is not a performance, usability or security defect.

How BVA could have prevented it:
BVA tests at and around each boundary, because defects cluster there. For a field valid from 1 to 100:

Test value	Purpose
0	just below the minimum
1	minimum
2	just above the minimum
99	just below the maximum
100	maximum
101	just above the maximum

Executing these would have exposed the crash during testing. The team suspects improper coverage, and BVA gives systematic, measurable coverage of boundaries with a small number of high-value tests. Combined with equivalence partitioning, it also covers the valid and invalid classes.

Question 9

A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.

Answer: A defect repository stores defect descriptions, affected modules, severity, root causes, steps to reproduce, fixes, and regression tests. Testers should analyze recurring patterns and add targeted cases for previously faulty areas. If past releases repeatedly failed at coupon expiry, include expired, just-expired, and boundary-time coupon scenarios in regression suites. Reusing this evidence improves coverage, prevents recurrence, supports trend analysis, and helps prioritize high-risk modules.

Concept: A defect repository (defect database) is a central store of all defects with details such as ID, summary, module, severity, priority, type, root cause, steps to reproduce, status and fix.

How it supports test case design:

1. **Identifying defect-prone areas:** modules with many past defects get more and deeper test cases.
2. **Creating new test cases from past defects:** each defect becomes a test case, so the same defect cannot recur unnoticed.
3. **Analyzing defect types:** if boundary and validation defects are frequent, add more BVA and negative tests.
4. **Finding test gaps:** defects found late or in production show which test cases were missing.
5. **Building the regression suite:** fixed defects are added to regression packs.
6. **Training and checklists:** common defects feed review checklists.

Role in improving quality:

- It stops repeated defects across releases.
- It supports root-cause analysis and defect prevention.
- It supplies metrics (defect density, trends, leakage) for decisions.
- It preserves organizational knowledge when staff change.

Example: Release 1 had a date-format defect in reports. The repository entry leads to test cases for multiple date formats in every later release, so the defect no longer repeats.

Question 10

An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.

Answer: This is a functional defect because the withdrawal behavior does not meet the required transaction rules. The ATM must check available balance and limits, debit the account once, dispense the correct cash, update records, and handle network or cash-dispensing failures safely. Test sufficient and insufficient balances, limit boundaries, duplicate requests, timeouts, reversals, and concurrent transactions. End-to-end and integration tests with clear expected results could have detected the defect.

Description: A **functional defect** occurs when the software does not do what the functional requirements specify. The ATM withdrawal requirement is: verify the PIN, check the balance, dispense cash, deduct the amount, and print the receipt. Here the transaction logic is handled incorrectly.

Possible forms of the defect:

- The balance is deducted but cash is not dispensed (no rollback).
- The amount is deducted twice.
- The daily limit is not enforced.
- Withdrawal succeeds when the balance is insufficient.

Why it is a functional defect: the system's behavior deviates from the specified business logic. It is not a performance or usability issue. It was not detected because the relevant scenarios were not tested.

How it could have been prevented:

- **Requirement reviews** that define all transaction rules and failure paths.
- **Test design techniques:** decision tables for combinations of PIN status, balance and limits, state-transition testing for the transaction states, and BVA for amounts (the limit, the limit + 1, a balance of exactly the amount).
- **Negative and failure tests:** network failure mid-transaction, cash dispenser jam, power loss.
- **Code reviews and unit tests** of the transaction module.
- **Integration testing** with the bank server, and **regression testing** after fixes.

Question 11

A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.

Answer: Defect clustering means that a relatively small number of modules often contain a large share of defects. Use defect history, code complexity, change frequency, and business criticality to identify those modules. Allocate deeper testing, more regression cases, and reviews to high-defect areas while retaining baseline coverage for all modules. This risk-based allocation uses limited time effectively and increases the chance of finding important defects.

Concept: Defect clustering is a testing principle which says a small number of modules usually contain most of the defects. It reflects the **Pareto principle**: about 80% of defects come from about 20% of modules. Reasons include complexity, frequent changes, poorly understood code and developer inexperience.

Identifying testing priorities:

1. Collect defect data per module from the repository.
2. Rank modules by defect count, density and severity.
3. Combine this with risk, complexity and change frequency.
4. Divide modules into high, medium and low priority.

Testing plan:

- **High-defect modules:** more test cases, deeper techniques, code reviews, experienced testers, more frequent regression.
- **Medium modules:** standard coverage.
- **Low-defect modules:** basic coverage, with periodic checks.

How it improves efficiency:

- Effort goes where defects are most probable, so more defects are found per hour.
- Time and cost are saved, since not every module is tested equally.
- Release risk drops because the riskiest areas are well covered.
- Developers can refactor or redesign the troublesome modules.

Caution: Low-defect modules must not be ignored, and clusters can shift between releases, so the data must be updated regularly.

Question 12

A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.

Answer: This is a non-functional performance defect, specifically a load or scalability problem. Record the workload, response time, throughput, error rate, and resource use. Reproduce it with load and stress tests, then investigate bottlenecks such as slow database queries, insufficient connection pools, or CPU and memory limits. Correct classification directs the team to performance analysis rather than only checking functional outputs, helping resolve the underlying cause.

Classification: This is a **non-functional defect**, specifically a **performance defect** (response time, throughput or scalability). The application works correctly with a single user, so no functional requirement fails. The failure appears under volume.

Defect classification techniques:

- **By type:** functional, performance, usability, security, compatibility, interface.
- **By severity:** critical, major, minor, trivial. Here it is major or critical, as users cannot complete tasks.
- **By priority:** high, medium, low.
- **By phase of origin:** requirements, design, coding.
- **By root cause:** inefficient queries, no caching, memory leaks, poor thread handling, inadequate infrastructure.

How classification helps resolution:

- The defect is routed to the right specialists (performance engineers, DBAs, architects).
- The right test is applied: load, stress, endurance and spike testing with tools such as JMeter.
- Prioritization is easier, and critical defects are fixed first.
- Trend analysis shows recurring categories, which leads to process improvement.
- Management gets clear metrics and reports.

Example: Profiling shows an unindexed database query slowing the page under 5,000 users. Once classified as a performance defect, the DB team adds an index and the page recovers.

Question 13

A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.

Answer: Review historical defects and map them to modules, root causes, fixes, and related requirements. Convert important defect reproductions into permanent regression tests, prioritize tests for frequently changed or defect-prone areas, and include tests for connected modules that may be affected. Automate stable, repeatable cases where practical. This reuses proven knowledge, protects against regressions, improves coverage of past failure patterns, and reduces repeated test-design effort.

Concept: Regression testing confirms that changes have not broken existing functionality. Running everything every time is costly, so historical defect data guides selection and prioritization.

How to reuse the data:

1. Convert fixed defects into regression test cases. Every defect found in an earlier release gets a test that confirms it stays fixed.
2. Identify high-risk modules: modules with frequent defects get more regression tests (defect clustering).
3. Prioritize by severity and frequency: tests linked to critical or repeating defects run first.
4. Analyze defect types: if boundary defects dominate, add boundary tests to the suite.
5. Link defects to code changes: a change in a module triggers the tests tied to that module's past defects.
6. Remove obsolete or redundant tests that never detect defects.
7. Feed automation: automate the stable, high-value cases.

Improvement in coverage: historically fragile areas are covered, and fixes are verified in every cycle.

Improvement in efficiency: the suite becomes **risk-based** and smaller, so execution time drops, feedback is faster and effort is better spent. Fewer defects leak into production.

Question 14

During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.

Answer: Testers act as quality partners by clarifying requirements, discussing interfaces and data contracts, reviewing designs, agreeing on expected behavior, and sharing reproducible defect reports. Participate in planning and stand-ups, use a common defect tracker, and communicate severity and evidence clearly. Early collaboration exposes assumptions about APIs, data formats, and error handling before integration, reducing misunderstandings and costly rework.

Problem: Poor communication leads to misunderstanding of interfaces, data formats, responsibilities and requirements, and these show up as integration defects.

Role of a tester:

- Participate in requirement, design and interface reviews to understand and question assumptions.

- Clarify interface contracts: data types, formats, units, error codes, sequence of calls.
- Share test cases and acceptance criteria early so developers know the expected behavior.
- Write clear defect reports with steps, expected vs actual results, logs and screenshots, in a neutral, constructive tone.
- Take part in daily stand-ups, sprint planning and retrospectives in agile teams.
- Maintain traceability and a shared defect repository.
- Plan integration tests early, including stubs, drivers and test data.
- Act as a bridge between business, developers and operations.

How early involvement helps:

- Everyone shares one understanding before coding begins, so interface mismatches are avoided.
- Issues are raised at design time, not during integration.
- Trust grows, and the relationship becomes collaborative instead of adversarial.

Question 15

A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.

Answer: Perform root-cause analysis rather than repeatedly patching symptoms. Review recurring defects, improve requirements and design reviews, introduce coding standards and static analysis, strengthen unit tests, update checklists, and train the team where needed. Add regression tests for each confirmed defect and monitor recurrence metrics. These preventive actions address the process or design weakness that created the defect, reducing repeat failures and improving maintainability and reliability.

Concept: Defect detection finds defects after they are injected.

Defect prevention removes the causes so they are not injected. It is the long-term solution for recurring defects.

Techniques to apply:

1. Root cause analysis (RCA): use a fishbone diagram, 5-Whys and Pareto analysis on past defects to find the underlying causes.
2. Reviews and inspections: formal peer reviews of requirements, design and code.
3. Coding standards and checklists built from common defects.
4. Training: target the skill gaps that the defect data reveals.
5. Process improvement: update templates, definitions of done and review procedures.
6. Static analysis tools and automated unit tests run in CI.
7. Lessons learned and retrospectives after each release.
8. Defect trend analysis and metrics to monitor effectiveness.
9. Requirement clarity and prototyping to avoid misunderstanding.

How prevention improves overall quality:

- Fewer defects are introduced, so fewer need fixing.
- Rework, cost and schedule pressure drop.
- Releases become more stable and predictable.
- Customer satisfaction and team morale improve.
- Quality becomes built-in instead of inspected-in.

Example: Repeated null-pointer defects are traced to a missing input check. A coding standard, a static analysis rule and developer training remove the whole class of defect.

CODE OF CONDUCT

I, M Sandhya(192312421), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: sandhya

Date: 23-10-2026

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 marks)	Level 3 – Proficient (4 marks)	Level 2 – Developing (2–3 marks)	Level 1 – Beginning (0–1 mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand